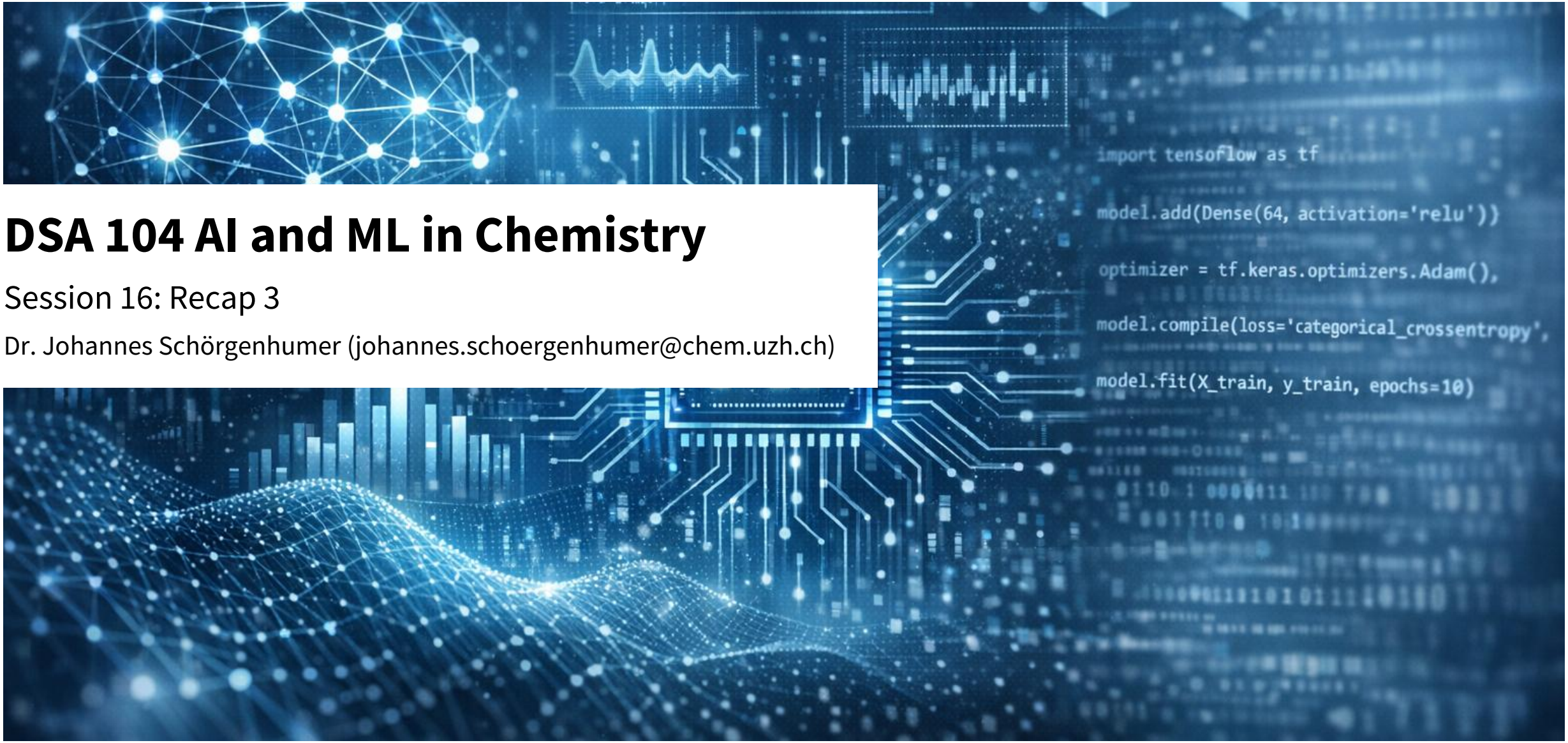
Universität Zürich[UZH]

Department of Chemistry



DSA 104 AI and ML in Chemistry

Session 16: Recap 3

Dr. Johannes Schörgenhumer (johannes.schoergenhumer@chem.uzh.ch)

Points for today

DSA105 Check-In

Discuss Assignment 5 (assignment 6 next week)

Follow up: Pretrained models and transfer learning (exercise from session 15)

Light content: Time series

Light content 2: Reinforcement learning

DSA105 Check-In

Anyone else to sign up?

Continued: Using foundation models in practice

Preparation:

1) In VSC code, after starting the environment, run: *uv remove transformers --optional transformers*
2) Then, run: *uv add transformers --optional transformers* and *uv sync --extra transformers*
3) Tell me the version number (need to specify the lowest in the project.toml in the repo)

Exercise:

— Use a pretrained foundation model as a fixed encoder to generate features from SMILES in the ESOL dataset.

— Build a small regression model on top to predict solubility

— Compare the performance with small ML models trained on fingerprints or descriptors

Some links on transfer learning in different life science domains:

- Transfer Learning in Biological and Health Care: https://link.springer.com/chapter/10.1007/978-981-16-0811-7_5
- Transfer learning for a foundational chemistry model: https://pubs.rsc.org/en/content/articlelanding/2024/sc/d3sc04928k
- Transfer Learning from Simulation to Experimental Data: NMR Chemical Shift Predictions: https://pubs.acs.org/doi/10.1021/acs.jpclett.1c00578
- Transfer learning across different chemical domains: virtual screening of organic materials with deep learning models pretrained on small molecule and chemical reaction data: https://link.springer.com/article/10.1186/s13321-024-00886-1
- Knowledge-reused transfer learning for molecular and materials science: https://www.sciencedirect.com/science/article/abs/pii/S2095495624004145
- Transfer learning enables predictions in network biology: https://www.nature.com/articles/s41586-023-06139-9
- A robust transfer learning approach for high-dimensional linear regression to support integration of multi-source gene expression data: https://journals.plos.org/ploscompbiol/article?id=10.1371%2Fjournal.pcbi.1012739

Papers also provided in the repo!

Time Series

A sequence of observations ordered in time, e.g. sensor data, website traffic, …:
$\boldsymbol{x_1, x_2, x_3, x_4, \dots x_t}$

Key characteristic:

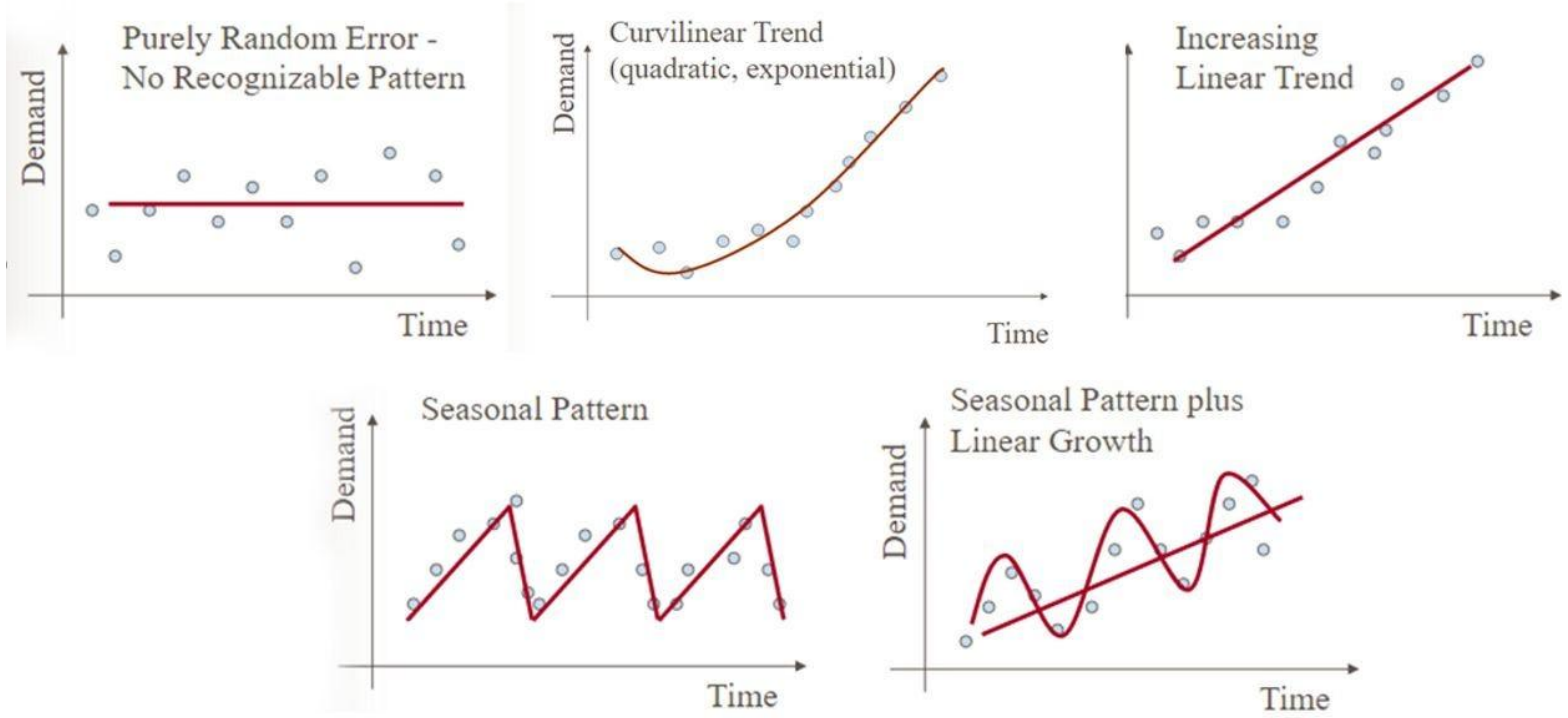
— **Temporal dependency**: past observations influence future values

Typical goals:

— Forecasting

— Anomaly detection

— Classification of sequences



What would be examples in the life science domain?

https://storage.googleapis.com/dgrinjknqvomoe/what-are-the-types-of-time-series-forecasting-methods.html

Time Series – key characteristics

Key properties:

— **Trend**: Long term increase/decrease

— **Seasonality**: Repeating patterns in regular intervals

— **Cycles**: Longer irregular fluctuations

— **Noise**: Random variation not explained by the signal

Model: $y_t = Trend + Seasonality + Noise$

Stationarity for a time series is given if…

— Mean is constant

— Variance is constant

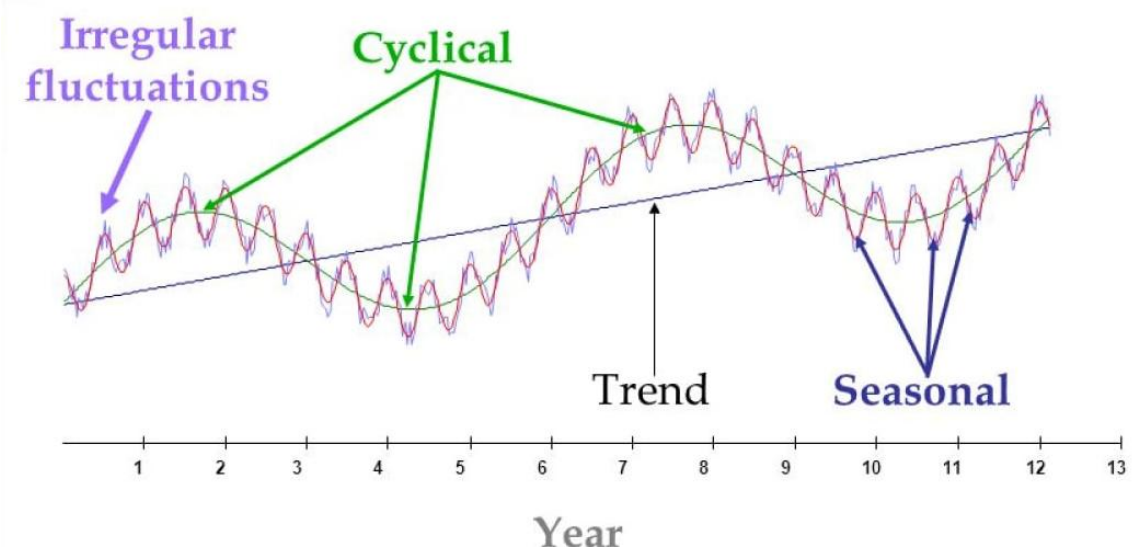
Typical transformations:

— Differencing

— Detrending ($y'_t = y_t - y_{t-1}$)

— Log transformations

— Seasonal decomposition



https://www.ford-team.de/4-common-machine-learning-data-transforms-for-time-series-forecasting/

Time Series – how to treat the data:

Python datetime objects may be very useful!

Feature engineering:

— Lag features: past observations used as predictors ($x_{t-1}, x_{t-2}, x_{t-3}$)

— Rolling statistics: e.g. rolling mean ($rolling\ mean_t = \frac{1}{k}\sum_{i=0}^{k-1} x_{t-i}$)

— Calendar features: e.g. hour, day of the month

Data splits:

— Random splits don't work

— Use temporal slice instead!

Models:

— Classical models, e.g. AutoRegressive, Moving Average

— ML: Treat as regression!

— DL: NN, Transformers

First look at Time Series

Have a look at the provided notebook (*timeseries.ipynb*) and…

…discuss what the individual steps do

…find some acceptable parameters

…how can you actually implement forecast of the next value? (Recursive vs. direct forecasting!)

Reinforcement Learning

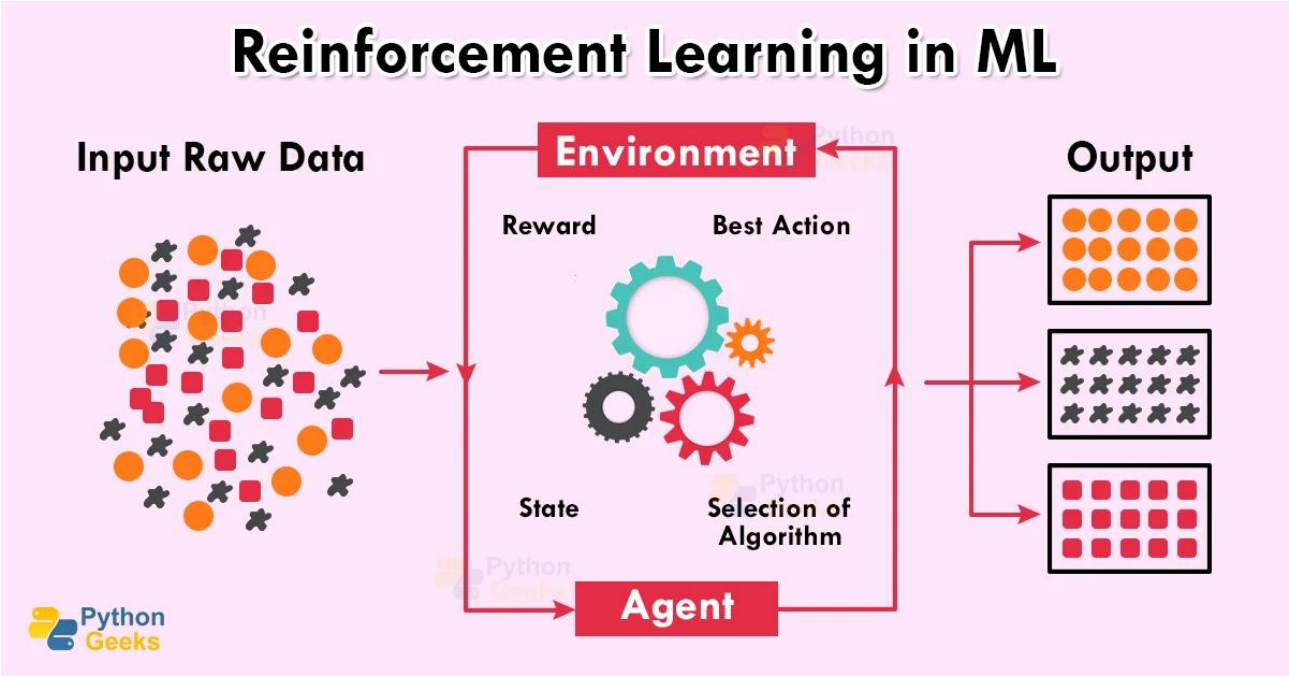
Key idea: an **agent learns how to act in an environment** by interacting with it and **receiving feedback in the form of rewards**.

Compare different types of ML:

— Supervised learning: labelled data, clear objective

— Unsupervised learning: unlabelled data, pattern recognition

— **Reinforcement learning (RL): no labelled data, but explicit objective via reward signal**



https://pythongeeks.org/reinforcement-learning-in-machine-learning/

Reinforcement Learning: Key concepts

Key components:

— **Agent**: the learner (model, robot, …)

— **Environment**: everything to interact with

— **State**: representation of current situation

— **Action**: Choices available to agent

— **Policy**: agent's behaviour

— **Reward**: feedback signal

Core idea:

1. **Observe the current state** of the environment
2. Agent takes an **action**
3. **Environment responds** and transitions to a new state
4. Agent receives **reward or penalty**
5. **Update strategy** (= policy)
6. Repeat and improve

Reinforcement Learning: Objective

The agent aims to **maximise expected cumulative reward**, also called the **return**, e.g.

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \ \ldots$$

…with $\boldsymbol{\gamma}$ as **discount factor** (importance of future rewards) and $\boldsymbol{r_t}$ the **reward** for the action in the **current** state.

Here (Markov Decision Process), the future **depends only on the presence, not the full history**.

This encourages:

— Long-term planning

— Delayed gratification

— Strategic behavior

Reinforcement Learning: Value functions

Value functions estimate “how good” states or actions are.

— **State Value** Functions:

— Expected discounted future reward when starting in state sss and following policy π\piπ

Expectation

$$\underbrace{V^{\pi}(s)}_{\text{The value of state } s \text{ when the agent follows policy } \pi.} = \mathbb{E}[G_t | s_t = s]$$

— **Action Value** Functions:

— Which actions lead to the highest long-term reward?

$$Q^{\pi}(s, a) = \mathbb{E}[G_t | s_t = s, a_t = a]$$

Reinforcement Learning: Exploration vs. Exploitation

A fundamental challenge and **unavoidable trade-off** in RL:

— **Exploitation**: choose best-known action

— **Exploration**: try new actions to gain knowledge

Common strategies to solve this :

— ε-greedy (ε ~ ratio of exploration/exploitation)

— Softmax action selection (probabilistic)

— Upper Confidence Bound (UCB)

Reinforcement Learning: Classes of RL Algorithms

- **Value-based** methods:
  - Learn value functions
  - Policy derived indirectly

- **Policy-based** methods:
  - Learn policy directly
  - Useful for continuous actions

- **Actor-Critic methods:**
  - Combination of:
    - Actor: learns policy
    - Critic: learns the value function

- **Deep Reinforcement Learning**: Deep-Q-Networks, AlphaGo, Robotics…

Reinforcement Learning: Random Overview of some applications

- Robotics: navigation, walking, handling of objects…
- Process control: adaptive control in manufacturing plants, automated storage management, …
- Autopilots, e.g. autonomous driving
- Business & Finances: evaluating business strategies or financial plans
- Game playing: Chess, Go, tic-tac-toe, …
- **Life Sciences:**
  - Medicine: Adaptive treatment planning
  - Chemistry: Reaction prediction
  - Chemistry, drug development (DD): Molecule design (e.g. catalysts, drug candidates)
  - Biology/Biochemistry: Protein engineering ad design
  - Lab Automation: Experimental design, plate designers

Generative catalyst design
Symposium: May 27!

Bayesian optimisers in Lab Automation:
Symposium: May 27!

Reinforcement Learning: Applicability

Useful when:

— Decisions are sequential

— Outcomes are delayed

— There are trade-offs over time

— Exploration is expensive but valuable

Not worth it in case of:

— Pure prediction tasks (supervised learning)

— Small datasets with no feedback loop (classical statistics)

— Single-step decisions (regression/classification)

Reinforcement Learning: Minimalistic example

Have a look at the minimal RL example RL-Bandit.ipynb. This is RL in its purest (and most reduced) form.